Defination:- Design and develop a small, self-contained full-stack mini-project of the student's choice (e.g., a poll/voting app, expense split calculator, URL shortener, quiz app, or bookmark manager) that integrates React (or Next.js), a Node.js/Express backend, and TypeScript.

Code:-

(1) questions.ts:-

```
export const questions = [
  {
    id: 1,
    question: "Which language is used to create web pages?",
    options: ["HTML", "Python", "Java", "C++"],
    answer: 0
  },
  {
    id: 2,
    question: "Which technology is used to build user interfaces?",
    options: ["React", "MongoDB", "Node.js", "Express"],
    answer: 0
  },
  {
    id: 3,
    question: "Which runtime is used to execute JavaScript on the server?",
    options: ["Node.js", "HTML", "CSS", "React"],
    answer: 0
  },
  {
    id: 4,
    question: "Which framework is used for creating REST APIs in our project?",
    options: ["Express", "React", "Next.js", "CSS"],
    answer: 0
  },
  {
    id: 5,
    question: "Which language are we using for type safety?",
    options: ["TypeScript", "HTML", "CSS", "SQL"],
    answer: 0
  }
];
```

(2) quizRoutes.ts:-

```
import { Router } from "express";
import { questions } from "../data/questions";

const router = Router();

// Function to shuffle an array
function shuffleArray<T>(array: T[]): T[] {
  const shuffled = [...array];

  for (let i = shuffled.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));

    [shuffled[i], shuffled[j]] = [
      shuffled[j],
      shuffled[i]
    ];
  }

  return shuffled;
}

// Get quiz questions
router.get("/questions", (req, res) => {
  const quizQuestions = questions.map((question) => {
    const shuffledOptions = shuffleArray(question.options);

    return {
      id: question.id,
      question: question.question,
      options: shuffledOptions
    };
  });

  res.json(quizQuestions);
});

// Submit quiz
```

```
router.post("/submit", (req, res) => {
  const { answers } = req.body;

  if (!Array.isArray(answers)) {
    return res.status(400).json({
      message: "Answers must be provided as an array."
    });
  }

  if (answers.length !== questions.length) {
    return res.status(400).json({
      message: `Please answer all ${questions.length} questions.`
    });
  }

  let score = 0;

  answers.forEach(
    (
      answer: {
        questionId: number;
        selectedAnswer: string;
      }
    ) => {
      const question = questions.find(
        (q) => q.id === answer.questionId
      );

      if (!question) {
        return;
      }

      const correctAnswer =
        question.options[question.answer];

      if (answer.selectedAnswer === correctAnswer) {
        score++;
      }
    }
```

```
  );

  const percentage = Math.round(
    (score / questions.length) * 100
  );

  const status =
    percentage >= 60 ? "PASS" : "FAIL";

  res.json({
    score,
    total: questions.length,
    percentage,
    status,
    message: "Quiz completed successfully"
  });
});

// API status
router.get("/status", (req, res) => {
  res.json({
    status: "online",
    totalQuestions: questions.length,
    message: "Student Quiz API is running successfully"
  });
});

export default router;
```

(3) server.ts:-

```
import express from "express";
import cors from "cors";
import quizRoutes from "./routes/quizRoutes";

const app = express();

const PORT = 5000;
```

```
// Middleware
app.use(cors());
app.use(express.json());

// Home route
app.get("/", (req, res) => {
  res.send("Student Quiz API is running");
});

// Quiz routes
app.use("/api", quizRoutes);

// Start server
app.listen(PORT, () => {
  console.log(
    `Server running at http://localhost:${PORT}`
  );
});
```

(4) page.tsx:-

```
"use client";

import { useEffect, useState } from "react";
import styles from "./page.module.css";

interface Question {
  id: number;
  question: string;
  options: string[];
}

interface SelectedAnswer {
  questionId: number;
  selectedAnswer: string;
}

interface Result {
  score: number;
```

```
  total: number;
  percentage: number;
  status: string;
  message: string;
}

export default function Home() {
  const [questions, setQuestions] = useState<Question[]>([]);
  const [answers, setAnswers] = useState<
    SelectedAnswer[]
  >([]);

  const [currentQuestion, setCurrentQuestion] =
    useState(0);

  const [result, setResult] =
    useState<Result | null>(null);

  const [loading, setLoading] =
    useState(true);

  const [error, setError] =
    useState("");

  // Load questions
  useEffect(() => {
    fetch("http://localhost:5000/api/questions")
      .then((response) => {
        if (!response.ok) {
          throw new Error(
            "Failed to load questions"
          );
        }

        return response.json();
      })
      .then((data: Question[]) => {
        setQuestions(data);
        setLoading(false);
```

```
    })
    .catch(() => {
      setError(
        "Unable to connect to the quiz server."
      );

      setLoading(false);
    });
}, []);

// Select answer
const selectAnswer = (
  selectedAnswer: string
) => {
  const questionId =
    questions[currentQuestion].id;

  const updatedAnswers = [...answers];

  const existingAnswer =
    updatedAnswers.findIndex(
      (answer) =>
        answer.questionId === questionId
    );

  const newAnswer: SelectedAnswer = {
    questionId,
    selectedAnswer
  };

  if (existingAnswer !== -1) {
    updatedAnswers[existingAnswer] =
      newAnswer;
  } else {
    updatedAnswers.push(newAnswer);
  }

  setAnswers(updatedAnswers);
};
```

```
// Get selected answer
const getSelectedAnswer = () => {
  if (!questions[currentQuestion]) {
    return "";
  }

  const questionId =
    questions[currentQuestion].id;

  const answer = answers.find(
    (item) =>
      item.questionId === questionId
  );

  return answer
    ? answer.selectedAnswer
    : "";
};

// Next question
const nextQuestion = () => {
  if (
    currentQuestion <
    questions.length - 1
  ) {
    setCurrentQuestion(
      currentQuestion + 1
    );
  }
};

// Previous question
const previousQuestion = () => {
  if (currentQuestion > 0) {
    setCurrentQuestion(
      currentQuestion - 1
    );
  }
```

```
};

// Submit quiz
const submitQuiz = async () => {
  if (answers.length !== questions.length) {
    setError(
      "Please answer all questions before submitting."
    );

    return;
  }

  try {
    setError("");

    const response = await fetch(
      "http://localhost:5000/api/submit",
      {
        method: "POST",
        headers: {
          "Content-Type":
            "application/json"
        },
        body: JSON.stringify({
          answers
        })
      }
    );

    if (!response.ok) {
      throw new Error(
        "Submission failed"
      );
    }

    const data: Result =
      await response.json();

    setResult(data);
```

```
  } catch {
    setError(
      "Unable to submit the quiz."
    );
  }
};

// Restart quiz
const restartQuiz = () => {
  setAnswers([]);
  setCurrentQuestion(0);
  setResult(null);
  setError("");
  setLoading(true);

  fetch("http://localhost:5000/api/questions")
    .then((response) =>
      response.json()
    )
    .then((data: Question[]) => {
      setQuestions(data);
      setLoading(false);
    })
    .catch(() => {
      setError(
        "Unable to connect to the quiz server."
      );

      setLoading(false);
    });
};

// Loading screen
if (loading) {
  return (
    <main className={styles.page}>
      <div className={styles.loading}>
        <div
          className={styles.spinner}
```

```
        ></div>

        <p>Loading quiz...</p>
      </div>
    </main>
  );
}

// Error screen
if (error && questions.length === 0) {
  return (
    <main className={styles.page}>
      <div className={styles.errorBox}>
        <h2>Connection Error</h2>

        <p>{error}</p>

        <p>
          Make sure the Express backend is
          running on port 5000.
        </p>
      </div>
    </main>
  );
}

// Result screen
if (result) {
  const passed =
    result.percentage >= 60;

  return (
    <main className={styles.page}>
      <div className={styles.resultCard}>

        <div
          className={`${styles.resultIcon} ${
            passed
              ? styles.passIcon
```

```
    : styles.failIcon
 }`}
>
 {passed ? "✓" : "✗"}
</div>

<h1>Quiz Completed!</h1>

<p
 className={
  styles.resultMessage
 }
>
 {result.message}
</p>

<div
 className={styles.scoreCircle}
>
 <span>
  {result.percentage}%
 </span>
</div>

<h2>
 {result.score} /{" "}
 {result.total}
</h2>

<p
 className={
  styles.scoreText
 }
>
 Your final score
</p>

<h2
 className={
```

```
          passed
            ? styles.passText
            : styles.failText
        }
       >
        {passed
          ? "PASS"
          : "FAIL"}
       </h2>

       <p className={styles.passRule}>
        Minimum Passing Percentage: 60%
       </p>

       <button
        className={
          styles.primaryButton
        }
        onClick={restartQuiz}
       >
        Restart Quiz
       </button>
      </div>
    </main>
  );
}

// Current question
const question =
  questions[currentQuestion];

const progress =
  ((currentQuestion + 1) /
    questions.length) *
  100;

return (
  <main className={styles.page}>
```

```
<div
 className={
  styles.quizContainer
 }
>

 {/* Header */}

 <header
  className={styles.header}
 >
  <div>

   <p
    className={styles.tag}
   >
    FULL-STACK MINI PROJECT
   </p>

   <h1>
    Student Quiz App
   </h1>

   <p
    className={
     styles.subtitle
    }
   >
    Test your knowledge and
    see your score instantly.
   </p>

  </div>

  <div
   className={
    styles.questionCount
   }
  >
```

```
    {currentQuestion + 1} /{" "}
    {questions.length}
   </div>

  </header>

  {/* Progress */}

  <div
   className={
    styles.progressArea
   }
  >

   <div
    className={
     styles.progressBar
    }
   >
    <div
     className={
      styles.progress
     }
     style={{
      width: `${progress}%`
     }}
    ></div>
   </div>

   <p>
    Question{" "}
    {currentQuestion + 1}{" "}
    of{" "}
    {questions.length}
   </p>

  </div>

  {/* Error message */}
```

```
{error && (
 <div
  className={
   styles.smallError
  }
 >
  {error}
 </div>
)}

{/* Question */}

<section
 className={
  styles.questionCard
 }
>

 <h2>
  {question.question}
 </h2>

 <div
  className={
   styles.options
  }
 >

  {question.options.map(
   (option, index) => (
    <button
     key={option}
     className={`${
      styles.option
     } ${
      getSelectedAnswer() ===
      option
       ? styles.selected
```

```
            : ""
          }`}
          onClick={() =>
            selectAnswer(
              option
            )
          }
        >

          <span
            className={
              styles.optionLetter
            }
          >
            {String.fromCharCode(
              65 + index
            )}
          </span>

          <span>
            {option}
          </span>

        </button>
      )
    )}

  </div>

</section>

{/* Navigation */}

<div
  className={
    styles.navigation
  }
>
```

```
<button
 className={
  styles.secondaryButton
 }
 onClick={
  previousQuestion
 }
 disabled={
  currentQuestion === 0
 }
>
 ← Previous
</button>

{currentQuestion ===
questions.length - 1 ? (
 <button
  className={
   styles.submitButton
  }
  onClick={
   submitQuiz
  }
 >
  Submit Quiz
 </button>
) : (
 <button
  className={
   styles.primaryButton
  }
  onClick={
   nextQuestion
  }
 >
  Next →
 </button>
)}
```

```
      </div>

      {/* Footer */}

      <footer
        className={styles.footer}
      >
        Built with Next.js,
        TypeScript, Node.js &
        Express
      </footer>

    </div>

  </main>
 );
}
```

(5) page.module.css:-

```
.page {
  min-height: 100vh;
  padding: 40px 20px;
  background: #f4f7fb;
  display: flex;
  justify-content: center;
  align-items: center;
}

.quizContainer {
  width: 100%;
  max-width: 850px;
}

.header {
  background: white;
  padding: 32px;
  border-radius: 18px;
  display: flex;
  justify-content: space-between;
```

```
  align-items: flex-start;
  box-shadow: 0 5px 25px rgba(0, 0, 0, 0.07);
  margin-bottom: 22px;
}

.tag {
  font-size: 12px;
  font-weight: 700;
  letter-spacing: 1.5px;
  margin: 0 0 8px;
  color: #2563eb;
}

.header h1 {
  margin: 0;
  font-size: 34px;
  color: #111827;
}

.subtitle {
  margin: 8px 0 0;
  color: #667085;
}

.questionCount {
  font-size: 18px;
  font-weight: 700;
  padding: 10px 16px;
  border-radius: 10px;
  background: #f1f5f9;
  color: #344054;
}

.progressArea {
  margin-bottom: 22px;
}

.progressArea p {
  margin: 8px 0;
```

```
  color: #667085;
  font-size: 14px;
}

.progressBar {
  height: 8px;
  background: #e5e7eb;
  border-radius: 10px;
  overflow: hidden;
}

.progress {
  height: 100%;
  background: #2563eb;
  border-radius: 10px;
  transition: width 0.3s ease;
}

.questionCard {
  background: white;
  padding: 35px;
  border-radius: 18px;
  box-shadow: 0 5px 25px rgba(0, 0, 0, 0.07);
}

.questionCard h2 {
  margin-top: 0;
  font-size: 24px;
  line-height: 1.4;
  color: #111827;
}

.options {
  display: flex;
  flex-direction: column;
  gap: 14px;
  margin-top: 28px;
}
```

```
.option {
  width: 100%;
  border: 2px solid #e5e7eb;
  background: white;
  padding: 17px;
  border-radius: 12px;
  display: flex;
  align-items: center;
  gap: 15px;
  cursor: pointer;
  text-align: left;
  font-size: 16px;
  transition: 0.2s;
  color: #111827;
}

.option:hover {
  border-color: #2563eb;
  transform: translateY(-1px);
}

.selected {
  border-color: #2563eb;
  background: #eff6ff;
}

.optionLetter {
  width: 34px;
  height: 34px;
  min-width: 34px;
  border-radius: 50%;
  background: #f1f5f9;
  display: flex;
  align-items: center;
  justify-content: center;
  font-weight: 700;
}

.selected .optionLetter {
```

```
  background: #2563eb;
  color: white;
}

.navigation {
  display: flex;
  justify-content: space-between;
  margin-top: 22px;
  gap: 15px;
}

.primaryButton,
.secondaryButton,
.submitButton {
  border: none;
  padding: 14px 24px;
  border-radius: 10px;
  font-size: 15px;
  font-weight: 700;
  cursor: pointer;
  transition: 0.2s;
}

.primaryButton {
  background: #2563eb;
  color: white;
}

.primaryButton:hover {
  background: #1d4ed8;
}

.secondaryButton {
  background: white;
  color: #344054;
  border: 1px solid #d0d5dd;
}

.secondaryButton:hover:not(:disabled) {
```

```
  background: #f8fafc;
}

.secondaryButton:disabled {
  opacity: 0.5;
  cursor: not-allowed;
}

.submitButton {
  background: #16a34a;
  color: white;
}

.submitButton:hover {
  background: #15803d;
}

.footer {
  text-align: center;
  margin-top: 25px;
  color: #667085;
  font-size: 13px;
}

.loading {
  text-align: center;
  background: white;
  padding: 50px;
  border-radius: 18px;
  box-shadow: 0 5px 25px rgba(0, 0, 0, 0.07);
}

.spinner {
  width: 35px;
  height: 35px;
  border: 4px solid #e5e7eb;
  border-top-color: #2563eb;
  border-radius: 50%;
  animation: spin 0.8s linear infinite;
```

```
  margin: auto;
}

.errorBox {
  background: white;
  padding: 40px;
  border-radius: 18px;
  text-align: center;
  max-width: 500px;
  box-shadow: 0 5px 25px rgba(0, 0, 0, 0.07);
}

.smallError {
  background: #fef2f2;
  color: #b91c1c;
  border: 1px solid #fecaca;
  padding: 12px 16px;
  border-radius: 10px;
  margin-bottom: 18px;
  text-align: center;
  font-weight: 600;
}

.resultCard {
  background: white;
  padding: 45px;
  border-radius: 20px;
  text-align: center;
  width: 100%;
  max-width: 500px;
  box-shadow: 0 5px 25px rgba(0, 0, 0, 0.08);
}

.resultIcon {
  width: 70px;
  height: 70px;
  margin: auto;
  border-radius: 50%;
  display: flex;
```

```
  align-items: center;
  justify-content: center;
  font-size: 36px;
  font-weight: bold;
}

.passIcon {
  background: #dcfce7;
  color: #16a34a;
}

.failIcon {
  background: #fee2e2;
  color: #dc2626;
}

.resultCard h1 {
  margin: 20px 0 8px;
  color: #111827;
}

.resultMessage {
  color: #667085;
}

.scoreCircle {
  width: 130px;
  height: 130px;
  border-radius: 50%;
  border: 10px solid #2563eb;
  display: flex;
  justify-content: center;
  align-items: center;
  margin: 30px auto 15px;
}

.scoreCircle span {
  font-size: 28px;
  font-weight: 800;
```

```
  color: #111827;
}

.resultCard h2 {
  margin: 10px 0 0;
  font-size: 28px;
}

.scoreText {
  color: #667085;
  margin-bottom: 10px;
}

.passText {
  color: #16a34a;
  font-weight: 800;
}

.failText {
  color: #dc2626;
  font-weight: 800;
}

.passRule {
  color: #667085;
  font-size: 14px;
  margin-bottom: 25px;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

@media (max-width: 600px) {
  .page {
    padding: 20px 12px;
  }
```

```
  .header {
    padding: 22px;
    flex-direction: column;
    gap: 15px;
  }

  .header h1 {
    font-size: 27px;
  }

  .questionCard {
    padding: 22px;
  }

  .questionCard h2 {
    font-size: 20px;
  }

  .navigation {
    flex-direction: column;
  }

  .primaryButton,
  .secondaryButton,
  .submitButton {
    width: 100%;
  }

  .resultCard {
    padding: 30px 20px;
  }
}
```

(6) globals.css:-

```
@import "tailwindcss";

:root {
```

```
  --background: #ffffff;
  --foreground: #171717;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
}

@media (prefers-color-scheme: dark) {
  :root {
    --background: #0a0a0a;
    --foreground: #ededed;
  }
}
* {
  box-sizing: border-box;
}

html,
body {
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, Helvetica, sans-serif;
}
```

(7) layout.tsx:-

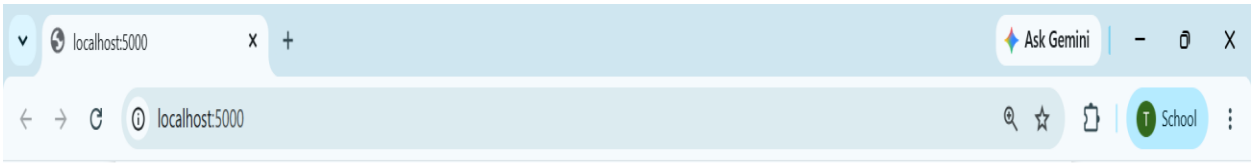
```
import type { Metadata } from "next";
import "./globals.css";

export const metadata: Metadata = {
  title: "Student Quiz App",
  description:
```

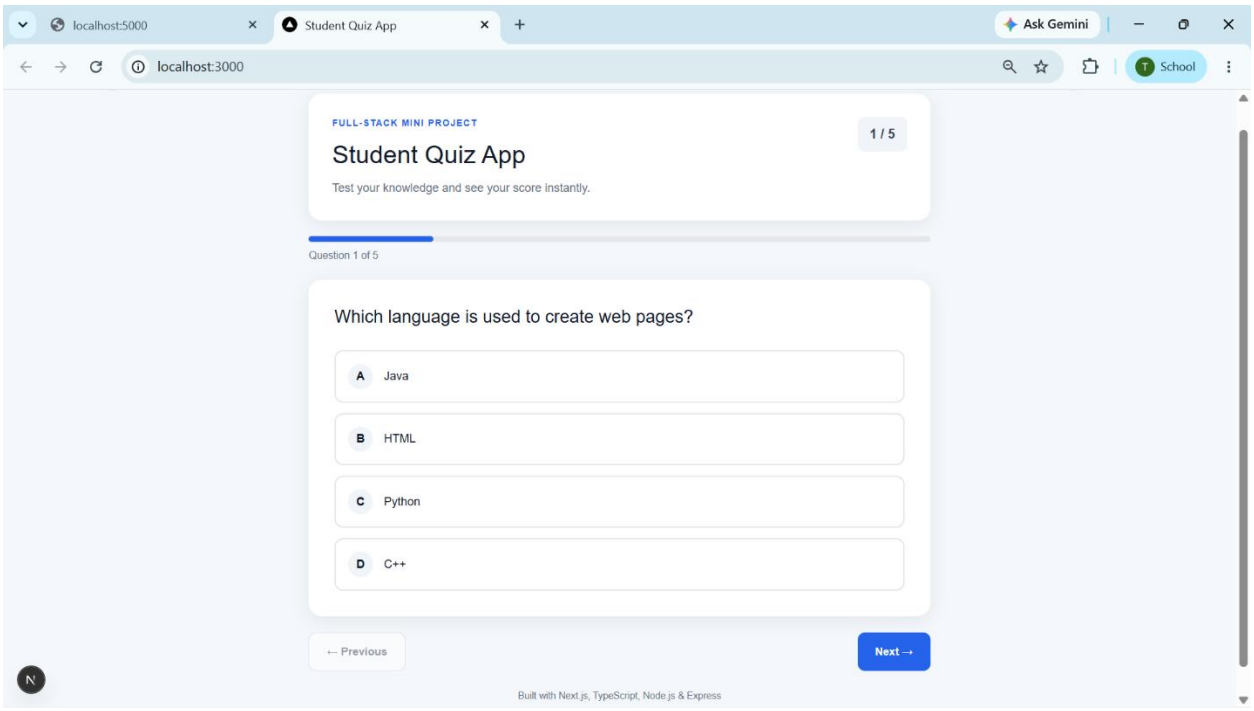
```
    "Full-stack Student Quiz Application"
};

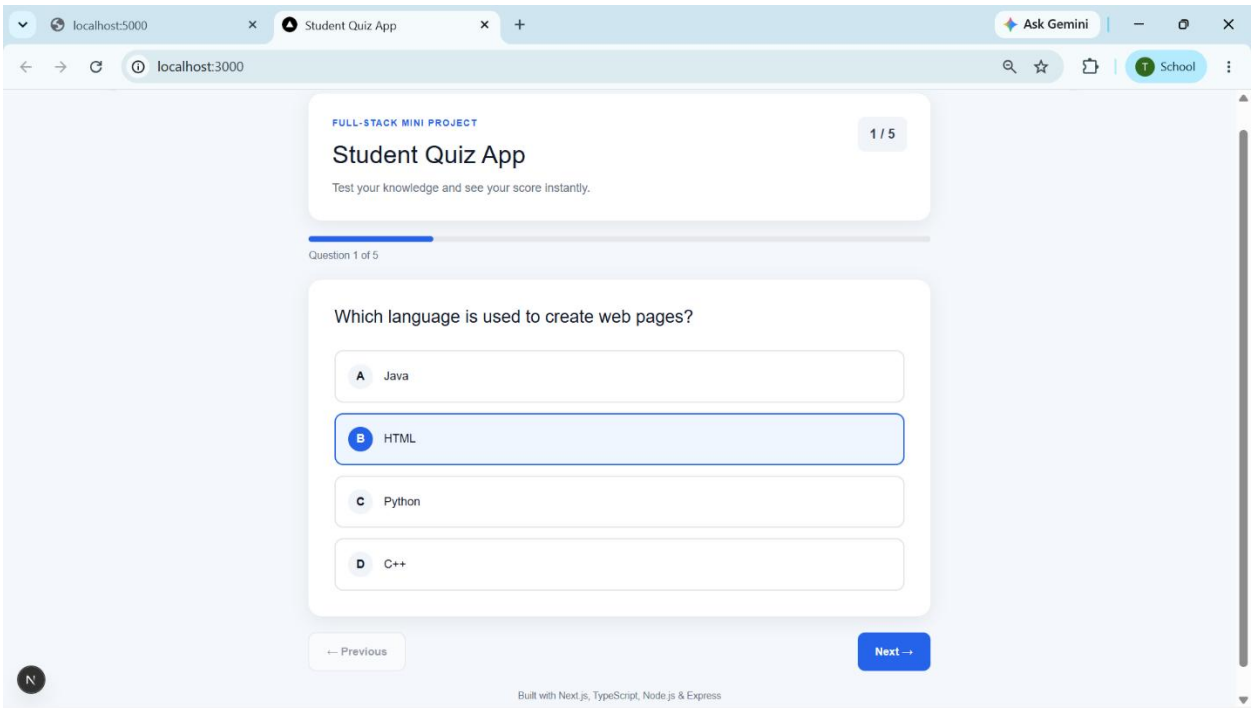
export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="en">
      <body>{children}</body>
    </html>
  );
}
```

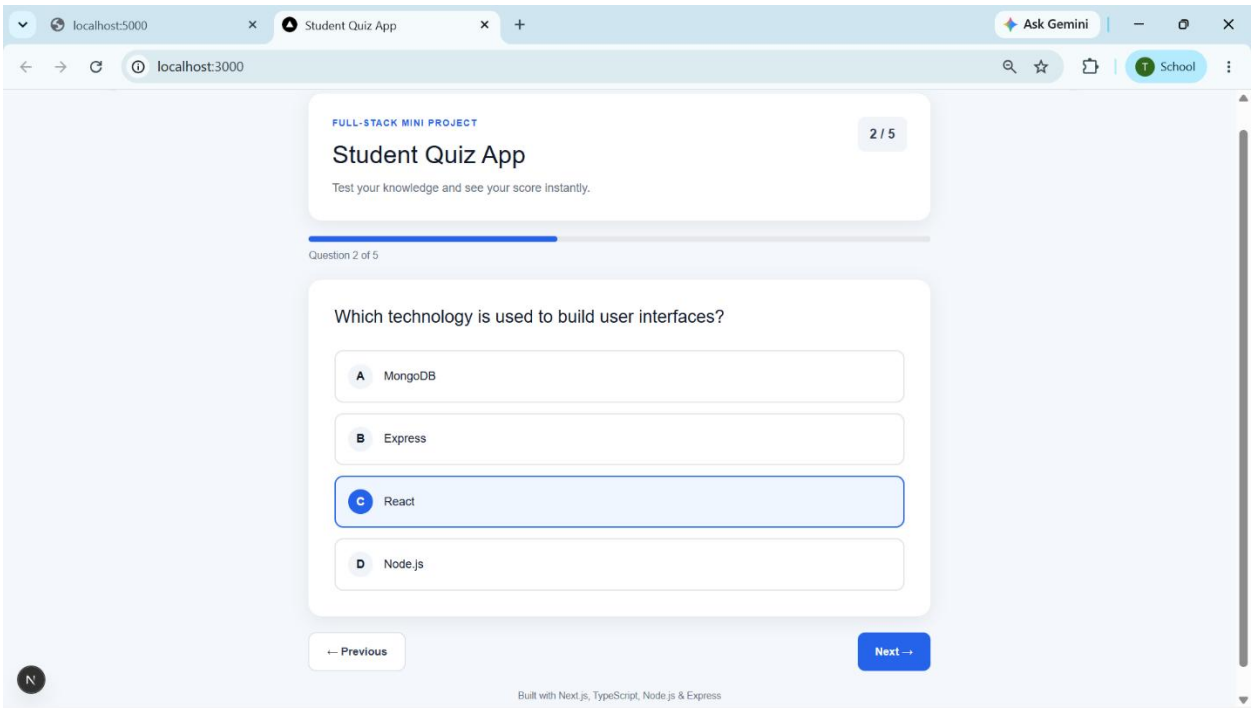
Output Screenshot:-

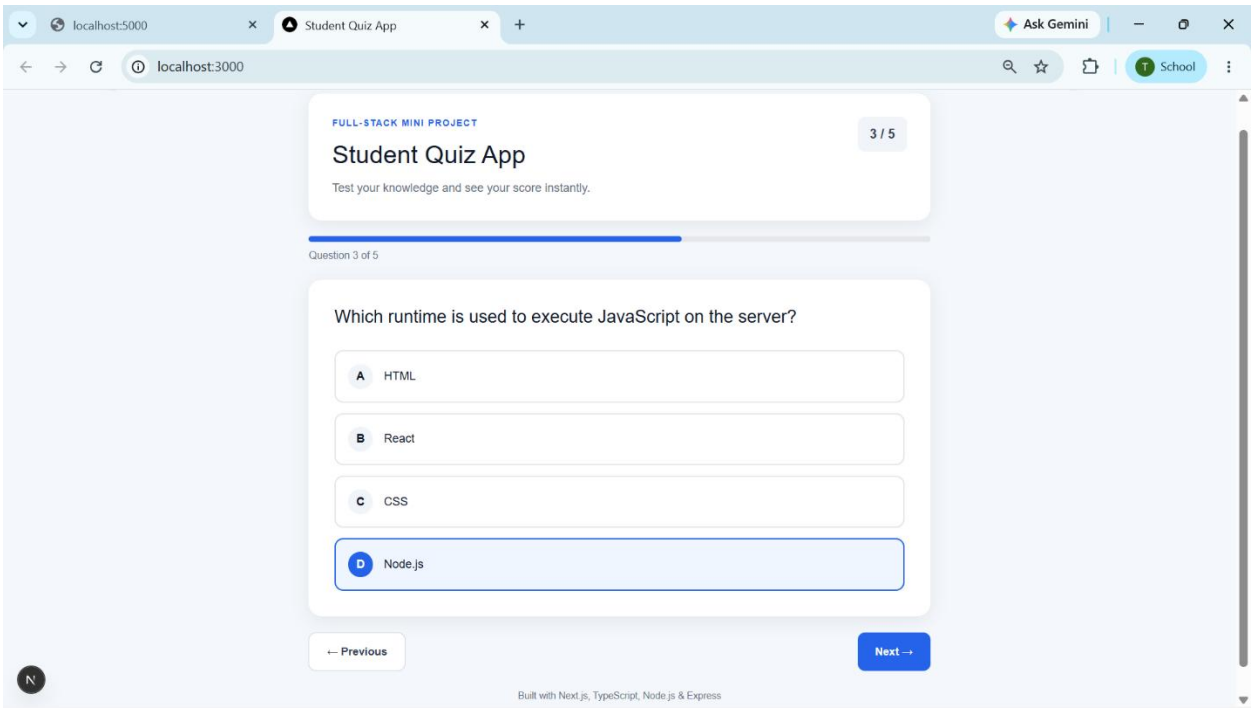


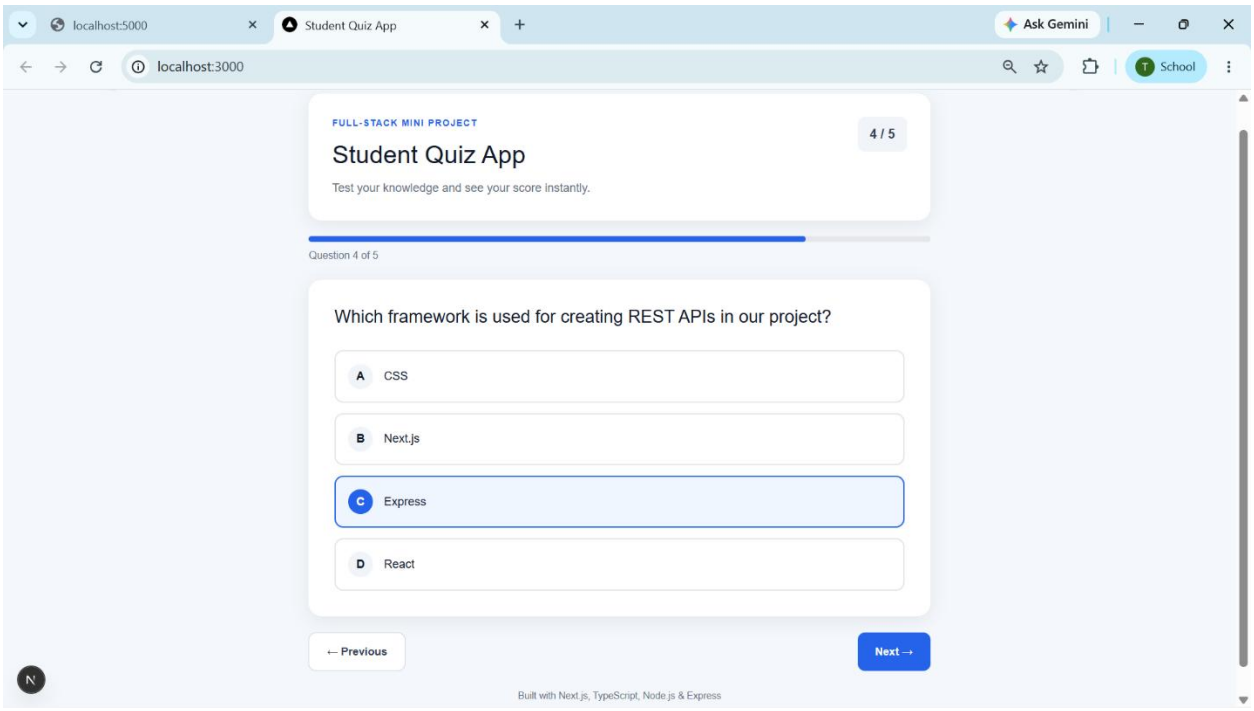
Student Quiz API is running

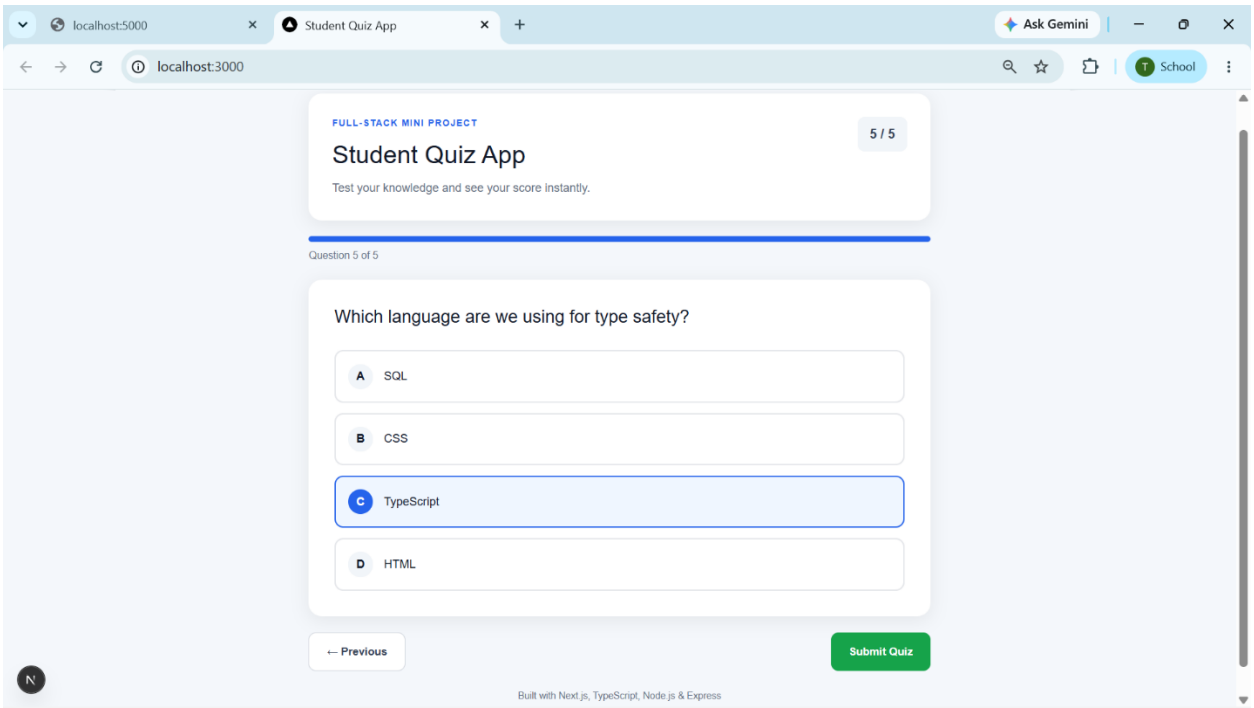

localhost:5000
Student Quiz App
Ask Gemini
localhost:3000
School
FULL-STACK MINI PROJECT
1 / 5
Student Quiz App
Test your knowledge and see your score instantly.
Question 1 of 5
Which language is used to create web pages?
A Java
B HTML
C Python
D C++
← Previous
Next →
Built with Next.js, TypeScript, Node.js & Express

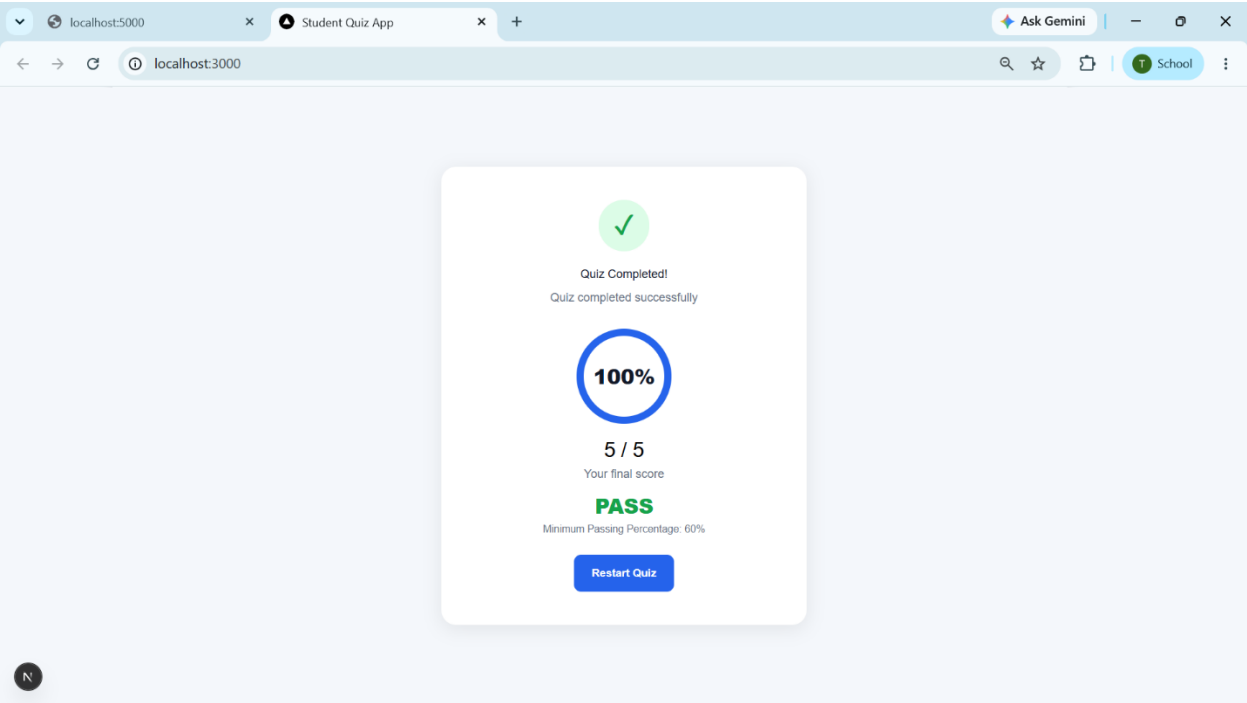

localhost:5000
Student Quiz App
Ask Gemini
localhost:3000
School
FULL-STACK MINI PROJECT
1 / 5
Student Quiz App
Test your knowledge and see your score instantly.
Question 1 of 5
Which language is used to create web pages?
A Java
B HTML
C Python
D C++
← Previous
Next →
Built with Next.js, TypeScript, Node.js & Express

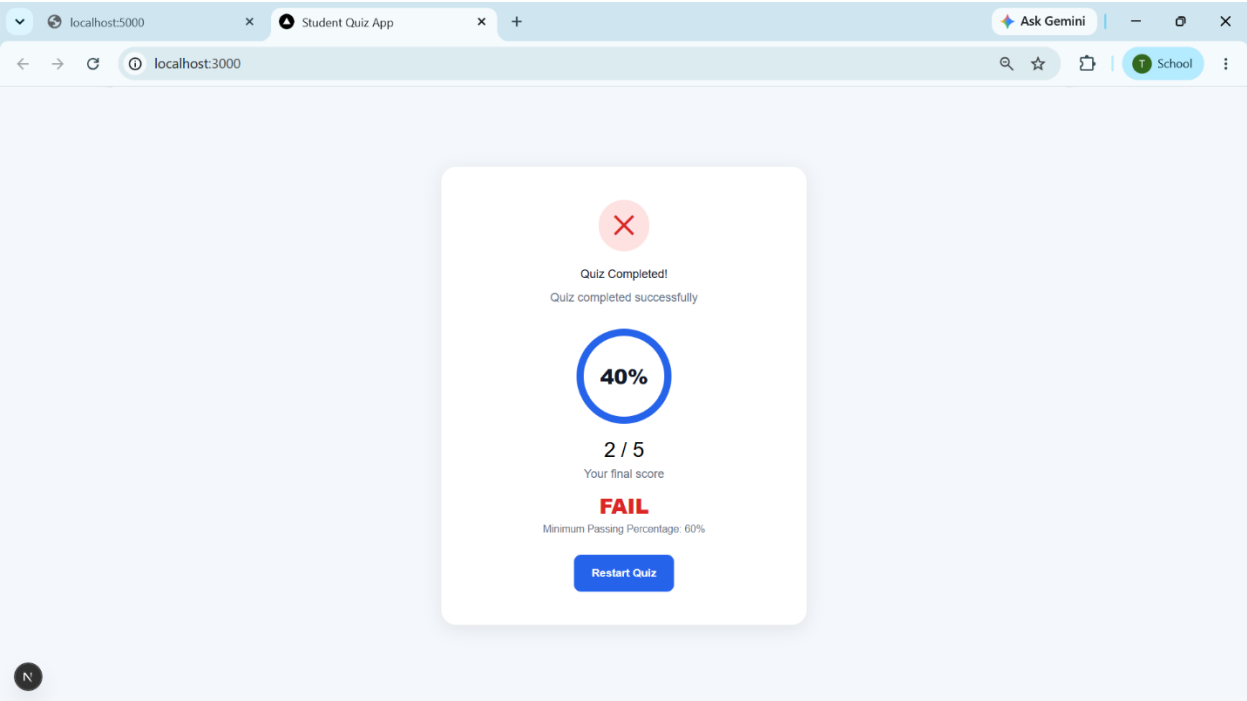

localhost:5000
Student Quiz App
Ask Gemini
localhost:3000
School
FULL-STACK MINI PROJECT
2 / 5
Student Quiz App
Test your knowledge and see your score instantly.
Question 2 of 5
Which technology is used to build user interfaces?
A MongoDB
B Express
C React
D Node.js
← Previous
Next →
Built with Next.js, TypeScript, Node.js & Express


localhost:5000
Student Quiz App
Ask Gemini
localhost:3000
School
FULL-STACK MINI PROJECT
3 / 5
Student Quiz App
Test your knowledge and see your score instantly.
Question 3 of 5
Which runtime is used to execute JavaScript on the server?
A HTML
B React
C CSS
D Node.js
← Previous
Next →
Built with Next.js, TypeScript, Node.js & Express


localhost:5000
Student Quiz App
Ask Gemini
localhost:3000
School
FULL-STACK MINI PROJECT
4 / 5
Student Quiz App
Test your knowledge and see your score instantly.
Question 4 of 5
Which framework is used for creating REST APIs in our project?
A CSS
B Next.js
C Express
D React
← Previous
Next →
Built with Next.js, TypeScript, Node.js & Express


localhost:5000
Student Quiz App
Ask Gemini
localhost:3000
School
FULL-STACK MINI PROJECT
5 / 5
Student Quiz App
Test your knowledge and see your score instantly.
Question 5 of 5
Which language are we using for type safety?
A SQL
B CSS
C TypeScript
D HTML
← Previous
Submit Quiz
Built with Next.js, TypeScript, Node.js & Express


localhost:5000
Student Quiz App
Ask Gemini
localhost:3000
School
Quiz Completed!
Quiz completed successfully
100%
5 / 5
Your final score
PASS
Minimum Passing Percentage: 60%
Restart Quiz


localhost:5000
Student Quiz App
Ask Gemini
localhost:3000
School
Quiz Completed!
Quiz completed successfully
40%
2 / 5
Your final score
FAIL
Minimum Passing Percentage: 60%
Restart Quiz